

Rapport de projet

Étude sur les liens entre start-ups et structures d'enseignement supérieur et de recherche

Quentin Ruel – Olivier Baudouin

Introduction :

Ce rapport présente les choix techniques et méthodologiques que nous avons faits dans le cadre de notre projet. L'objectif de ce dernier était d'analyser les données publiques relatives aux start-ups françaises liées à des structures d'enseignement supérieur et de recherche (ESR), et de produire un programme interactif permettant une analyse personnalisée.

Nous avons rédigé un script Python divisé en trois parties : l'importation et la préparation des données, le traitement à travers un ensemble de fonctions réutilisables et enfin le programme d'affichage interactif. Ce rapport détaille chacune de ces parties et justifie nos choix.

1. Importation et préparation des données :

Les données proviennent directement de la plateforme des données publiques française, via une URL de téléchargement. Cela nous a permis de charger le fichier sans avoir à télécharger les données manuellement, ce qui rend le chargement des données autonome et non dépendant d'un chemin d'accès propre à chaque utilisateur.

Les noms de colonnes d'origine étaient assez longs et pas toujours pratiques à manipuler. On a donc décidé de tous les renommer avec des noms plus courts et plus parlants (par exemple « nom », « date_crea », « esr_nom » ...). Cela nous a fait gagner beaucoup de temps par la suite.

Nous avons ensuite converti les colonnes dans les bons types. Par défaut, le package Pandas charge tout en « object », ce qui n'est pas toujours adapté.

- **Les dates :** Les colonnes de dates (date_crea, date_fermeture et date_obs) ont été converties en datetime avec `pd.to_datetime()`. On a utilisé le paramètre `errors = « coerce »` pour que les valeurs mal formatées deviennent des NaT. C'était indispensable pour pouvoir calculer les durées de vie des start-ups par la suite.
- **Le statut :** La colonne « statut » contenait les chaînes de caractères « active » et « inactive ». On a utilisé `.map()` pour les transformer en True et False, puis on a converti en type boolean (qui gère les valeurs manquantes). Cela nous a permis de filtrer plus facilement les start-ups fermées tout en gardant les valeurs manquantes telles quelles sans les confondre avec des « faux ».
- **Les colonnes numériques :** Nous voulions passer les colonnes siren, code_postal, depart_num et region_num en entier. Le problème, c'est que la colonne depart_num contenait les valeurs « 2A » et « 2B » pour la Corse, ce qui empêchait la conversion. On a donc choisi de remplacer ces valeurs par « 20 » (le numéro historique du département corse). Ensuite, on a utilisé le type `Int64` qui gère les types nullables. C'était nécessaire parce que certaines de ces colonnes contenaient des valeurs manquantes.
- **Les catégories :** Nous avons converti les colonnes actifs, region_char, et code_iso en type « category » car elles contenaient un nombre limité de valeurs différentes.
- **Les chaînes de caractères :** Nous avons converti toutes les autres colonnes en « string » pour éviter les comportements parfois imprévisibles du type « object ».

NB : Les colonnes `date_accompagnement` et `code_commune` n'ont pas été converties parce qu'elles prenaient certaines valeurs qui compliquaient grandement leur bonne conversion. Cela n'impactera pas le reste de notre programme parce qu'elles ne sont de toute façon pas conservées.

Une fois les colonnes converties dans un type satisfaisant, nous avons supprimé les colonnes qu'on n'allait pas utiliser dans la suite du programme. L'idée était d'alléger le `DataFrame` et optimiser l'exécution, pour que les traitements soient plus rapides. Cela ne change pas grand-chose dans notre cas parce que la base de données est relativement petite, mais pour une base plus importante cela serait primordial.

Nous avons par la suite exporté une version nettoyée du jeu de données dans un fichier CSV en utilisant un chemin relatif afin d'assurer la bonne exécution du code, peu importe l'utilisateur. Les chemins relatifs exportent l'objet dans le même dossier dans lequel est sauvegardé le script Python.

Au cours de l'exploration des données, nous avons remarqué que certaines cellules contenaient plusieurs valeurs séparées par des virgules. Par exemple, une start-up pouvait posséder plusieurs types d'actifs ou était liée à plusieurs structures ESR. Nous avons donc créé la fonction **`ensemble_valeurs(df,col)`** qui parcourt une colonne, découpe chaque cellule sur les virgules et retourne un ensemble de toutes les valeurs uniques. Cette fonction s'est avérée cruciale pour le reste du programme.

2. Traitement des données :

Cette partie regroupe toutes les fonctions qu'on a définies pour répondre aux questions posées par le sujet. L'idée générale était de créer des fonctions génériques et réutilisables, qu'on pourrait ensuite appeler dans le programme interactif selon les besoins.

Organisation des partenariats :

- **`startups_liees(df,nom_structure)`** : Cette fonction permet de récupérer toutes les start-ups liées à une structure ESR donnée en argument. Là encore, la difficulté venait des cellules multi-valuées : on ne pouvait pas se contenter d'un simple `==`. On a donc parcouru chaque cellule, découpé les éléments sur les virgules et construit un vecteur booléen pour filtrer le `DataFrame`. Il y avait peut-être plus optimisé, mais elle produit le résultat souhaité.
- **`st_partenaires(df,nom_structure)`** : Cette fonction identifie les structures ESR partenaires d'une structure donnée, c'est-à-dire celles qui partagent au moins une start-up en commun. Elle réutilise `startups_liees()` pour filtrer le `DataFrame`, puis `ensemble_valeurs()` pour extraire tous les noms de structures ESR du `DataFrame` filtré. On retire ensuite la structure d'origine avec `.discard()`.

Implantation géographique :

- **`nb_par_lieu(df,nom_col)`** : Cette fonction utilise `groupby()` et `nunique()` sur la colonne `siren` pour compter le nombre de start-ups uniques par zone géographique (donc soit par région, soit par département). On a choisi `nunique()` plutôt que `count()` au cas où une même start-up apparaissait sur plusieurs ligne, mais après vérification chaque numéro de `siren` est unique et nous aurions très bien pu utiliser `count()`. Cependant, nous obtenons le même résultat tel quel alors nous n'avons pas fait de modification.

- **impl_partenaires(df,nom_structure,nom_col)** : Cette fonction combine `startups_liees()` et `nb_par_lieu()` pour obtenir la répartition géographique spécifique à une structure. On trie les résultats par ordre décroissant pour que les plus représentés apparaissent en premier pour faciliter la lecture.

Start-ups ayant cessé leur activité :

- **startups_fermees(df)** : Cette fonction filtre les start-ups qui ont fermé (`statut == False`) et calcule leur durée de vie en soustrayant la date de création à la date de fermeture. Deux précautions ont été nécessaire : d'abord, on élimine les lignes où la date de fermeture est manquante (sinon le calcul plante). Ensuite, on retire les durées négatives qui n'ont pas de sens et fausseraient les moyennes. L'utilisation de `.copy()` évite un warning dans la console.
- **partenaires_fermees(df,nom_structure)** : Cette fonction combine la fonction `startups_fermees()` avec la fonction `startups_liees()`, et permet de récupérer les start-ups partenaires de la structure ESR donnée en argument qui ont cessé leur activité.
- **nb_fermees_region(df)** : Cette fonction n'était pas demandée par le sujet, mais nous avons choisi de la créer au cas où nous serions amenés à la réutiliser plus tard dans le script (ce qui n'a finalement pas été le cas). Elle renvoie le nombre de start-ups ayant fermé par région, sous forme de Series indexée par région. Elle combine la fonction `startups_fermees()` avec `nb_par_lieu()`, et permet donc de d'obtenir un comptage du nombre de start-up fermé par région.

Pour la question 8, à savoir celle qui nous demandait de calculer et de stocker dans des variables la durée de vie moyenne des start-ups fermées puis la durée de vie moyenne par région, nous n'étions pas certains de ce qu'on attendait réellement comme réponse.

La durée de vie moyenne globale n'a pas été un problème et a été stockée dans la variable `duree_vie_moy_fermees`. Nous avons juste à appliquer un `.mean()` à un DataFrame trié avec la fonction `startups_fermees()`.

En revanche pour les durées de vie moyenne par région, nous ne savions pas si nous devions créer 15 variables différentes correspondant à la durée de vie moyenne pour chaque région, ou tout regrouper dans un dictionnaire (ce qui est objectivement bien plus optimal). Nous avons donc laissé les deux options dans notre script.

Actifs produits par les entreprises :

- **avec_actif(df,nom_actif)** : Cette fonction reprend la structure de la fonction `startups_liees()` créée plus tôt, mais au lieu du noms de structure ESR nous testions la correspondance du type d'actif passé en argument. Elle renvoie donc un DataFrame dont les start-ups possèdent le type d'actif donné.
- **nb_par_categorie(df,nom_col,ensemble)** : Cette fonction permet de comptabiliser le nombre d'occurrences de chaque élément d'un ensemble donné au sein d'une colonne spécifique d'un DataFrame. Elle retourne un nouveau DataFrame contenant ces dénombrements.
- **nb_startup_actif(df,actif)** : Cette fonction filtre le DataFrame contenant le type d'actif donné avec l'appel de la fonction `avec_actif()`, puis compte le nombre de ligne dans celui-ci. Elle retourne le nombre de start-ups possédant l'actif passé en argument.

Nous avons ensuite calculé les proportions de chaque type d'actif. Dans un premier temps, créé un DataFrame regroupant toutes les start-ups possédant au moins un type d'actif. Ensuite, nous avons stocké dans un vecteur l'ensemble des valeurs que peut prendre la colonne « actifs » grâce à l'appel de la fonction `ensemble_valeurs()`. Puis pour chaque type d'actif on compte combien de start-ups le possèdent avec la fonction `nb_startup_actif()`, et on calcule la proportion correspondante en pourcentage, arrondie à une

décimale. Les résultats sont stockés dans le dictionnaire `dict_actif` où chaque clé est un type d'actif et la valeur associée est sa proportion.

Nous allons réutiliser exactement le même code dans la fonction à venir qui permet de tracer l'histogramme de répartition des actifs parmi les start-ups en possédant au moins un.

- **tracer_rep_actifs(df)** : Cette fonction permet de créer un histogramme illustrant la répartition des actifs parmi les start-ups en possédant au moins un. Comme annoncé précédemment, il reprend le dictionnaire `dict_actif`. En plus de cela, nous l'avons trié de façon décroissante par rapport à la valeur de proportion pour une meilleure lisibilité graphique. Nous avons augmenté la taille de la figure pour un meilleur rendu et ajouté les proportions au-dessus des barres de l'histogramme pour faciliter l'interprétation. Enfin, nous avons personnalisé le graphique avec des couleurs, des titres, et nous avons pivoté les noms des types d'actifs sur l'axe des abscisses pour une meilleure visibilité.

Nous en avons fini avec les questions du sujet. Avant de commencer notre troisième partie sur le programme interactif, nous avons défini une fonction similaire à `tracer_rep_actif()` mais qui prend un argument supplémentaire : un nom de structure ESR.

En effet, **tracer_rep_actifs_choix(df,nom_structure)** fonctionne exactement pareil que sa sœur aînée, mis à part qu'elle trie le DataFrame en fonction de la structure ESR donnée, et adapte le titre de l'histogramme en conséquence. Il est à noter qu'elle ne « rend » pas bien pour chaque structure ESR : en effet, si les startups liées à cette dernière ne comportent qu'un type d'actifs, l'histogramme ne comportera qu'une unique barre. Nous voulions néanmoins la créer pour permettre une analyse personnalisée complète dans notre programme.

3. Programme d'affichage interactif :

Avant de commencer la rédaction de notre programme final, nous trouvions intéressant de glisser dans ce rapport la première ébauche que nous avons réalisée. Celle-ci était fonctionnelle, mais ne comportait seulement les quelques fonctionnalités demandées par le sujet.

```
print("\nÉtude sur les start-ups liées à l'ESR français\n\n")
ens_structures_esr=sorted(ensemble_valeurs(df,"esr_nom"))

print(f"Voici les {len(ens_structures_esr)} structures ESR répertoriées dans la base de données :\n")
for e in ens_structures_esr:
    print(f" - {e}")

choix=input("\nSaisissez le nom de la structure ESR souhaitée : ")

print(f"\nRapport de : {choix}\n")
print(f"Nombre de startups liées : {len(startups_liees(df,choix))}, parmi lesquelles {len(partenaires_fermees(df,choix))} ont fermé.\n")
print(f"Répartition de ces {len(startups_liees(df,choix))} startups par département :")
repart_choix=impl_partenaires(df, choix, "depart_char")

df_choix=startups_liees(df,choix)
df_choix_fermees=nb_par_lieu(startups_fermees(df_choix,"depart_char").rename("nb_fermees"))
df_choix_all=pd.concat([repart_choix,df_choix_fermees],axis=1)

for departement in df_choix_all.index:
    if pd.isna(df_choix_all.loc[departement,"nb_fermees"]):
        if df_choix_all.loc[departement,"nb_fermees"]==1:
            print(f" - {departement} : {df_choix_all.loc[departement,'effectif']}, dont {int(df_choix_all.loc[departement,'nb_fermees'])} qui a fermé")
        else:
            print(f" - {departement} : {df_choix_all.loc[departement,'effectif']}, dont {int(df_choix_all.loc[departement,'nb_fermees'])} qui ont fermé")
    else:
        print(f" - {departement} : {df_choix_all.loc[departement,'effectif']}")

df_startups_fermees=startups_fermees(df_choix)
print(f"\nParmi les startups ayant fermé, la durée de vie moyenne est de {df_startups_fermees['duree_de_vie'].mean().days} jours.")
print("Si on s'intéresse aux durées de vie moyenne par département, nous avons :")
dvm_par_depart={}
for departement in df_choix_fermees.index:
    dvm_par_depart[departement]=df_startups_fermees[df_startups_fermees["depart_char"]==departement]["duree_de_vie"].mean()
print(f" - {departement} : {dvm_par_depart[departement].days} jours")

df_choix_actif=df_choix[df_choix["actifs"].notna()]
type_actif=ensemble_valeurs(df_choix_actif,"actifs")
dict_actif={}
for actif in type_actif:
    compte=nb_startups_actif(df_choix_actif,actif)
    proportion=round(compte/len(df_choix_actif) *100,1)
    dict_actif[actif]=proportion

dict_actif_trie = dict(sorted(dict_actif.items(),reverse=True, key=lambda x: x[1]))

print("\nRépartition des actifs parmi les startups partenaires en possédant au moins un :")
for actif, prop in dict_actif_trie.items():
    print(f" - {actif} : {prop}%")

print("\nSi vous souhaitez visualiser cela graphiquement, tapez \"OUI\". Sinon, tapez \"NON\".")
reponse=input("Votre réponse : ")

while reponse != "OUI" and reponse != "NON":
    print("Réponse invalide. Veuillez taper \"OUI\" ou \"NON\".")
    reponse = input("Votre réponse : ")

if reponse=="OUI":
    tracer_rep_actifs(df,choix)
    print("\nOuvrez la fenêtre \"Graphes\" afin de voir le graphique.")
else:
    print("\nComme vous voulez.\n\nFIN DU PROGRAMME")
```

Vous allez vous en rendre compte par la suite, mais on est bien loin de notre programme final. Même si on peut tout de même notifier quelques idées qu'on aura gardées ou adaptées ensuite.

Dans un premier temps, nous avons réservé une section dans laquelle nous avons créé trois fonctions.

La première, **menu(choix)**, est une succession d'instructions `print()` qui permet l'affichage d'un menu de choix proposé à l'utilisateur. Ce dernier aura le choix entre six sélections possibles. Les choix 1 à 4 correspondent aux choix proposés dans le sujet. En plus de ceux-ci, nous avons proposé l'affichage d'un rapport complet, reprenant toutes les analyses des choix 1 à 4, divisés en sections pour une bonne lisibilité. Cette possibilité nous a très vite semblée évidente, car si nous voulions une analyse complète sur une structure ESR, c'était fastidieux de rentrer 1, puis 2, puis 3, puis 4 dans le menu.

Le choix 6, lui, permet de retourner à la sélection de la structure ESR à analyser. L'idée nous est venue plus tard, lorsque nous testions les rendus pour différentes structures. En fait, devoir mettre fin au programme, tout resélectionner et relancer la compilation prenait du temps (et commençait à nous taper sur les nerfs). C'est pourquoi nous avons mis au point cette possibilité. Dans la pratique, cela permettrait à un utilisateur qui a envie de se renseigner sur plusieurs structure ESR d'enchaîner facilement et de manière fluide les rapports.

Enfin, l'option Autre permet de quitter le programme. L'utilisateur peut renseigner n'importe quoi dans son choix : un mot, une phrase, un chiffre (autre que 1 à 6), un espace ou même rien du tout, cela lui fera quitter le programme.

Les deuxièmes et troisièmes fonctions sont bien plus complexes, et non : nous ne les avons pas créées nous-même. Elles permettent de générer des versions PDF des rapports générés par notre script Python.

En fait, nous avons eu l'idée de donner la possibilité à l'utilisateur d'exporter les résultats du rapport complet sous forme de PDF afin qu'il puisse le consulter sur son ordinateur quand il le souhaite et qu'il ne soit pas obligé de revenir sur le script, recompiler le programme, re-rentre ses choix etc. Nous avons donc commencé à nous renseigner sur comment nous pouvions procéder, notamment grâce au package `fpdf`. Malheureusement, nous nous sommes très vite heurtés à 2 barrières : la complexité de la création d'une fonction qui marche, et la qualité du rendu de cette dernière (quand elle marchait).

Néanmoins, nous trouvions l'idée excellente et nous ne voulions pas la retirer de notre programme. C'est pourquoi on a fait le choix assumé de s'aider d'une IA Gen pour nous aider à créer des rapports sous format PDF dont le rendu est propre et satisfaisant.

Nous avons donc créé 2 fonctions :

- **generer_pdf()**, qui permet la création d'un rapport global (nous y reviendrons plus tard) au format PDF. Elle est la plus personnalisée des deux, car comme ses données ne changent pas, nous avons pu faire quelques commentaires d'analyses et optimiser l'espace pour un meilleur rendu.
- **generer_pdf_structure()**, qui elle permet la création d'un rapport ciblé sur la structure choisie par l'utilisateur. Comme les données changent selon le choix, son rendu l'est tout autant : parfois une image n'a pas suffisamment de place pour s'afficher sur la page, alors elle s'affiche sur la suivante mais du coup cela créé un large espace vide sur le rapport. Nous ne pouvons malheureusement pas calibrer le rapport pour « fitter » parfaitement aux données de chaque structure ESR, mais nous trouvons important de laisser cette possibilité à l'utilisateur car elle serait très pertinente en condition réelle, par exemple si un entrepreneur voudrait analyser ses différents choix d'affiliations.

Nous ne nous sommes bien entendu pas contenté de juste dire à l'IA Gen « nous avons besoin de mettre ça en PDF » avec un copier-coller de notre code. Nous avons fait une multitude de requête, d'ajustements, d'optimisations, de modifications etc. jusqu'à atteindre la forme finale présentée dans le script. Nous en sommes très satisfaits.

Comme nous n'avons pas codé de nos mains cette partie-là, nous n'allons pas l'expliquer en détail dans ce rapport. Néanmoins, nous en avons compris le fonctionnement global, et nous avons ajouté quelques commentaires sur ces fonctions sur notre script Python.

Nous arrivons enfin à la présentation de notre programme d'affichage interactif. Nous l'avons regroupé au sein d'une seule grande fonction **programme()** : au lieu de demander à l'utilisateur de sélectionner une centaine de lignes de code pour lancer le programme, il lui suffit d'appeler `programme()` en une seule ligne.

Le programme s'organise en deux phases : d'abord un rapport global sur l'ensemble des start-ups de la base, puis une phase interactive où l'utilisateur choisit une structure ESR et peut naviguer dans un menu d'actions.

Le rapport global :

Avant de plonger dans les rapports ciblés par structure ESR, il nous semblait essentiel de proposer une vue d'ensemble de notre base de données. L'idée est simple : permettre à l'utilisateur de prendre du recul, de saisir les grandes tendances du paysage des start-ups liées à l'ESR – le nombre de start-ups dans notre base de données, leur répartition géographique, leur taux de survie, les répartitions d'actifs - avant d'aller zoomer sur une structure en particulier. C'est un peu comme regarder la carte avant de choisir son chemin.

La première partie du programme affiche des statistiques générales : nombre total de start-ups, nombre de fermetures, répartition par région. Pour chaque région, on affiche l'effectif total et, si l'information est disponible, le nombre de start-ups ayant fermé. On a mis en place une condition qui teste s'il y a une ou plus d'une start-up ayant fermé afin d'avoir une cohérence dans le rendu (« 1 a fermé » vs « 5 ont fermé »).

On génère ensuite deux cartes de France avec le package GeoPandas : une pour la répartition des effectifs de start-ups par région, et l'autre pour le taux de fermeture. Ces deux cartes nous semblaient optimales pour analyser quelques dynamiques des start-ups en fonction de leur localisation, que nous avons reporté dans le rapport d'affichage.

Nous avons choisi de représenter uniquement la France métropolitaine parce que les effectifs des DROM étaient vraiment faibles comparativement et leur rendu n'était donc pas terrible. Nous sommes allés récupérer un fichier GeoJSON des régions françaises disponible sur GitHub pour la création des cartes, avec lequel nous avons fait une jointure avec nos données. On affiche ensuite le résultat avec un dégradé de couleur (jaune à rouge, code couleur très intuitif).

Le rapport global présente ensuite une analyse des durées de vie moyenne globale, puis en fonction des régions.

Ensuite, il réserve une partie à l'analyse des répartitions d'actifs. D'abord, la part d'entreprises en possession d'au moins un actif, puis leur part en fonction du type d'actif. Le programme appelle ensuite la fonction `tracer_rep_actifs()`, et représente sous forme d'histogramme cette répartition dans la fenêtre « Graphes ». Il en suit une courte analyse sur ce qu'on peut tirer de cette section.

À la fin de cette première phase, le programme propose à l'utilisateur d'exporter le rapport global en format PDF. L'utilisateur doit rentrer « OUI » ou « NON » selon son souhait. Nous avons fait en sorte que la casse et les espaces ne soient pas importants pour ce choix, car les réponses possibles ne sont pas complexes

(comme pour le choix de structure ESR qu'on verra ensuite). Si l'utilisateur rentre autre chose dans la console, le programme le lui notifie et repose la question. Si sa réponse est « OUI », le programme fait appel à la fonction `générer_pdf()` et crée le rapport global sous format PDF, qui sera stocké en utilisant le chemin relatif. Si sa réponse est « NON », le programme continue.

Le(s) rapport(s) ciblé(s) :

C'est la seconde et dernière partie du programme d'affichage interactif. Elle commence par une boucle `while` extérieure, qui permet le bon fonctionnement de l'option 6 du menu (à savoir la sélection d'une autre structure ESR). Le programme affiche ensuite les 260 structures ESR présentes dans la base de données et demande à l'utilisateur d'en saisir une, qui sera stockée dans la variable « choix ». Une nouvelle (petite) boucle `while` permet d'assurer la saisie correcte : tant que le choix ne correspond à aucune structure existante, on redemande. Cette fois-ci il faut respecter la casse lors de la saisie, parce que certains noms comportent des caractères à problèmes qu'on ne pouvait pas réellement gérer. C'était le moyen le plus fiable d'éviter les erreurs.

Le programme initialise ensuite plusieurs variables qui vont être utiles par la suite selon les choix d'analyse de l'utilisateur, ainsi que la variable « quitter » qui vaut `False` pour le moment et qui permettra de quitter le programme si elle vaut `True`.

On initialise une seconde boucle `while` qui permet à l'utilisateur d'enchaîner les analyses tant qu'il ne choisit pas l'option 6 ou ne quitte pas le programme. On affiche ensuite le menu d'actions à partir duquel l'utilisateur devra saisir l'action à effectuer, stockée dans une variable du même nom. À partir de là, le programme comporte 7 conditions `if` :

- **if action=='1'** : Le programme affiche l'analyse des partenariats de la structure ESR renseignée par l'utilisateur : le nombre de start-ups partenaires ainsi que leur nom et leur statut, le nombre de structures ESR liées ainsi que leur nom. Il n'y a pas de `break`, alors on reste dans la seconde boucle `while` et le menu est réaffiché.
- **elif action=='2'** : Le programme affiche la répartition par région des start-ups partenaires de la structure ESR renseignée par l'utilisateur. De plus, il affiche dans l'onglet « Graphes » une carte illustrant cette répartition. Le code couleur appliqué est le même que pour les cartes du rapport global (dégradé du jaune au rouge), et la couleur grisée correspondant aux départements dans lesquels il n'y a pas de start-up partenaire assure la bonne lisibilité de la carte. Il n'y a pas de `break`, alors on reste dans la seconde boucle `while` et le menu est réaffiché.
- **elif action=='3'** : Le programme affiche le nombre de start-ups partenaires ayant fermé ainsi que leur durée de vie moyenne, puis la durée de vie moyenne par département. Il prend bien en compte la possibilité qu'aucune start-up partenaire n'ait fermé grâce à une condition `if` intérieure. Il n'y a pas de `break`, alors on reste dans la seconde boucle `while` et le menu est réaffiché.
- **elif action=='4'** : Le programme affiche la part des start-ups partenaires en possession d'au moins un actif, ainsi que la répartition des types d'actifs parmi ces derniers. Il crée aussi un histogramme de cette répartition grâce à la fonction `tracer_rep_actifs_chois()` dans l'onglet « Graphes ». Le programme prend aussi en compte la possibilité qu'aucune start-up partenaire ne soit en possession d'actif grâce à une condition `if` intérieure. Il n'y a pas de `break`, alors on reste dans la seconde boucle `while` et le menu est réaffiché.
- **elif action=='5'** : Le programme affiche les résultats des actions 1 à 4, y compris les graphiques apparaissent dans l'onglet « Graphes ». À la fin du rapport complet, il propose à l'utilisateur d'en générer une version PDF : les conditions de passage sont exactement les mêmes que

précédemment dans le rapport global. Si l'utilisateur répond « OUI », une version PDF du rapport complet sur la structure ESR renseignée est générée et stockée en utilisant le chemin relatif en faisant appel à la fonction `générer_pdf_structure()`. Si « NON », le programme continue. Il n'y a pas de `break`, alors on reste dans la seconde boucle `while` et le menu est réaffiché.

- **elif action=='6'** : Le programme indique un retour à la sélection d'une structure ESR. Il y a un `break` : nous sortons de la seconde boucle `while`, ce qui nous permet de retourner à l'affichage des 260 structures ESR et à l'utilisateur de saisir un nouveau nom.
- **else** : Le programme dit au revoir à l'utilisateur et indique la fin du programme. La variable « quitter » prend la valeur `True`, et il y a un `break` : nous sortons de la seconde boucle `while`. La condition « `if quitter==True` » est validée : il y a un second `break`, qui nous fait sortir de la première boucle `while`. Le programme arrête son exécution.

Il y a ensuite une section dédiée à l'appel de la fonction `programme()`, afin de lancer son exécution facilement. Tous les packages nécessaires au fonctionnement du programme sont importés en début de script, immédiatement après le sommaire qui facilite la navigation dans le code.

Nous avons essayé de trouver un équilibre entre clarté du code (commentaires, noms de variables explicites, structure logique) et l'expérience utilisateur (validation des saisies, pluriels corrects, mise en page soignée des rapports). Le résultat est un outil fonctionnel qui permet d'explorer les données des start-ups liées à l'ESR français de manière interactive, et de générer des rapports personnalisés en PDF.